



Formation Développeur Full Stack

PROJET 6

Développeur Backend Java / Spring Boot

Concevez et développez le backend microservices du portail findMe pour la gestion des adressages urbain et périurbain.

Champ	Détail
Mis à jour le	Juillet 2026
Durée	4 semaines
Niveau	Intermédiaire / Avancé - Développeur Backend Java Junior
Pré-requis	Projet 4 - Frontend findMe (Nuxt 3) terminé et fonctionnel
Auteur	DHI Academy

1. Contexte et Scénario

Rappel du projet précédent

Il y a quatre semaines, vous avez rejoint GeoLink Africa, startup camerounaise spécialisée dans la digitalisation des adresses (Address-as-a-Service), pour concevoir le frontend du produit findMe. Le Projet 4 a permis de livrer une application Nuxt 3 moderne, mobile-first, accessible et orientée conversion, couvrant l'authentification, la gestion des adresses, la landing page, l'administration et le support client.

Cette application frontend est aujourd'hui terminée et fonctionnelle. Elle communique avec un Mock Server (JSON Server / Postman) qui simule les réponses de l'API en respectant scrupuleusement les contrats définis dans le cahier des charges du Projet 4 (routes, méthodes HTTP, formats JSON, codes de statut).

Nouveau contexte

Le Mock Server a rempli sa mission : il a permis de valider l'expérience utilisateur et de préparer une démonstration convaincante auprès d'investisseurs potentiels. La levée de fonds a été un succès. Mais le comité d'investissement pose désormais une condition claire à la suite du financement : la présentation, sous quatre semaines, d'un backend réel, sécurisé et prêt pour la production.

Le Mock Server, aussi utile ait-il été, présente des limites structurelles que la direction ne peut plus tolérer :

- Aucune règle métier n'est réellement appliquée (ex : la limite de 4 adresses par utilisateur n'est vérifiée que côté frontend).
- Aucune authentification réelle n'existe : il est aujourd'hui possible de se faire passer pour n'importe quel utilisateur, y compris un administrateur.
- Aucune donnée n'est réellement persistée : chaque redémarrage du Mock Server réinitialise l'ensemble du système.
- Il est impossible de démontrer la capacité de GeoLink à monter en charge (25 000 utilisateurs actifs et une ambition d'expansion en Afrique de l'Ouest).
- Aucune API documentée et stable n'est disponible pour les partenaires B2B (logistique, fintech, e-commerce) qui conditionnent leur intégration à une documentation OpenAPI fiable.

Les défis actuels

- ★ Absence totale de persistance réelle des données (utilisateurs, adresses, tickets support).
- ★ Aucune sécurité applicative : pas d'authentification robuste, pas de gestion des rôles et permissions (RBAC).
- ★ Les règles métier ne sont vérifiées que côté client, ce qui expose l'application à des contournements (l'API doit être la véritable ligne de défense).
- ★ Aucune traçabilité des actions (création, modification, suppression) ni horodatage fiable.
- ★ Aucune séparation claire des responsabilités entre les domaines métier : utilisateurs/authentification, adresses, administration/support.
- ★ Absence de documentation API exploitable par les équipes mobiles, les partenaires B2B et les futurs développeurs.
- ★ Aucune stratégie de tests automatisés garantissant la fiabilité du système avant mise en production.

Vision de la direction

Dans 4 semaines, GeoLink doit disposer d'un backend robuste, sécurisé, documenté et prêt pour la production, capable de remplacer intégralement le Mock Server, **sans qu'aucune modification ne soit nécessaire côté frontend.**

Contrainte non négociable : les contrats d'API (routes, méthodes HTTP, formats JSON, codes de statut) définis dans le Projet 4 doivent être strictement respectés. Le frontend Nuxt 3 doit pouvoir se brancher sur le backend réel en ne changeant qu'une seule variable d'environnement (l'URL de base de l'API).

Le backend doit également poser les bases d'une architecture en microservices, cohérente avec l'ambition de GeoLink de devenir une plateforme Address-as-a-Service exploitée par des dizaines de partenaires à travers l'Afrique de l'Ouest.

Mail de bienvenue du Directeur Technique

✉ Message

De : Lionel MBARGA - Directeur Technique, GeoLink Africa

À : Équipe Backend findMe

Objet : Bienvenue - Mission 4 semaines : Backend findMe

Bienvenue dans l'équipe backend,

Le frontend a fait ses preuves devant nos investisseurs, et j'en suis fier. Mais un frontend seul ne suffit pas à faire tourner une entreprise qui gère des dizaines de milliers d'adresses réelles. Aujourd'hui, tout repose sur un Mock Server. C'est terminé.

Votre mission pour ces 4 semaines est de concevoir et livrer un backend crédible, sécurisé et prêt pour la production, qui remplace entièrement le Mock Server.

Quelques points non négociables :

- ➔ Les contrats d'API définis dans le Projet 4 sont figés. Vous ne les renégociez pas, vous les implémentez fidèlement. Le frontend ne doit rien changer.
- ➔ La sécurité n'est pas une option. Nous venons de lever des fonds : nos investisseurs et nos partenaires B2B nous demanderont un audit de sécurité. JWT, rôles, permissions, hashing des mots de passe : tout doit être fait dans les règles.
- ➔ L'architecture doit être propre et modulaire. Je veux une séparation claire entre le domaine métier, les cas d'usage et l'infrastructure technique. Pas de logique métier dans les contrôleurs.
- ➔ Nous introduisons une architecture en microservices, mais restons pragmatiques : trois services métier et une passerelle suffisent pour ce projet. Pas de complexité inutile.
- ➔ Les tests ne sont pas du confort, ce sont une garantie. Je veux des tests unitaires sur la logique métier et des tests d'intégration sur les endpoints critiques.

Bon courage,

Lionel MBARGA

Directeur Technique - GeoLink Africa

Mail de cadrage technique - Lead Backend Engineer

Message

De : Samuel EWANE - Lead Backend Engineer, GeoLink Africa

À : Backend Developer

Objet : Cadrage technique - Backend findMe

Hello,

Lionel m'a demandé de te donner le cadrage technique complet. Voici ce que j'attends de toi, semaine par semaine.

Semaine 1 - Conception & Architecture

- Rédaction du dossier de conception (diagrammes, MCD, matrice RBAC, stratégie de sécurité).
- Découpage en 3 microservices (auth-service, address-service, admin-service) + API Gateway.
- Modélisation des bases PostgreSQL (une base par service) et premiers scripts Flyway.
- Squelette Maven multi-module et Docker Compose de base.

Semaine 2 - Auth-service & Address-service

- Implémentation complète de l'authentification (signup, signin, reset password) avec hashing BCrypt.
- Implémentation du CRUD adresses avec application stricte des règles métier (4 adresses max, unicité).
- Upload et gestion des images, pagination et filtrage conformes au contrat du Projet 4.

Semaine 3 - Sécurité, Intégration & Admin/Support

- Mise en place de Spring Security + JWT (access token, refresh token) et RBAC (@PreAuthorize).
- Développement de l'admin-service (vue agrégée utilisateurs/adresses, gestion du support client).
- Mise en place de l'API Gateway (routage + validation centralisée du JWT).
- Bascule du frontend du Mock Server vers le backend réel et validation bout en bout.

Semaine 4 - Qualité, Tests & Mise en production

- Tests unitaires et d'intégration (Testcontainers), couverture des cas critiques et des cas limites.
- Documentation OpenAPI complète pour chaque microservice, Actuator activé.
- Docker Compose final orchestrant l'ensemble du système.
- Rédaction du README et préparation de la soutenance.

Contraintes techniques

- Java 21 / Spring Boot 4 / Maven obligatoires.
- PostgreSQL + Flyway pour toute évolution de schéma.
- Clean Architecture / Architecture Hexagonale : domaine, use cases, infrastructure, web.
- Aucune modification des contrats d'API du Projet 4.

Documentation

Documente tout : README, arborescence, choix techniques. Je ne veux pas être le seul à comprendre ton code.

Samuel EWANE

Lead Backend Engineer - GeoLink Africa

2. Cahier des Charges Fonctionnel

Le backend est découpé en trois microservices métier autonomes, chacun disposant de sa propre base de données PostgreSQL (pattern database-per-service), et d'une API Gateway simplifiée assurant le routage et la validation centralisée du token JWT. Tous les contrats d'API (routes, formats, statuts) définis dans le Projet 4 doivent être respectés à l'identique.

2.1 Microservice Authentication & Utilisateurs (auth-service)

Objectif

Fournir une gestion sécurisée des comptes utilisateurs, l'émission et la validation des tokens JWT, ainsi que le contrôle d'accès basé sur les rôles (RBAC).

Ressource principale : User

- id, email (unique), motDePasseHashé, prénom, nom, rôle (USER, ADMIN, SUPPORT_AGENT)
- dateCréation, dateDerniereConnexion, statutCompte (actif/désactivé)

Endpoints attendus (contrat existant, non modifiable)

- POST /api/auth/signup - inscription
- POST /api/auth/signin - connexion, retourne access token + refresh token
- POST /api/auth/refresh - renouvellement du token d'accès
- POST /api/auth/logout - invalidation de session
- POST /api/auth/forgot-password - envoi d'un lien de réinitialisation
- POST /api/auth/reset-password - réinitialisation via token
- GET /api/users/me - profil de l'utilisateur connecté
- PUT /api/users/me - mise à jour du profil

Règles métier

- Chaque email est unique (contrainte au niveau base de données, pas seulement applicative).
- Le mot de passe doit contenir au minimum 8 caractères, une majuscule et un chiffre : la règle est revalidée côté serveur, même si le frontend la vérifie déjà.
- Les mots de passe sont hashés avec BCrypt ; aucun mot de passe en clair ne transite en base ni dans les logs.
- Le token de réinitialisation de mot de passe est à usage unique et expire après une durée définie.
- Les rôles disponibles sont USER, ADMIN et SUPPORT_AGENT, avec des permissions strictement différenciées.

Cas limites à gérer

- Inscription avec un email déjà utilisé → 409 Conflict avec message explicite.
- Identifiants invalides → 401 avec message générique (ne pas révéler si l'email existe).

- Lien de réinitialisation expiré ou déjà utilisé → 410 Gone / 400 Bad Request.
- Token JWT expiré ou invalide → 401 avec code d'erreur exploitable par le frontend.
- Accès à une ressource sans permission suffisante → 403 Forbidden.

2.2 Microservice Gestion des Adresses (address-service)

Objectif

Garantir la persistance, l'intégrité et l'application stricte des règles métier liées aux adresses, indépendamment de ce que fait ou ne fait pas le frontend.

Ressource principale : Address

- id, userId (propriétaire), pays, ville, quartier, rue, numéro, codePostal (optionnel)
- latitude, longitude, urlPhoto, dateCréation, dateModification

Endpoints attendus (contrat existant, non modifiable)

- GET /api/adresses - liste paginée, triée et filtrable des adresses de l'utilisateur connecté
- POST /api/adresses - création d'une adresse
- GET /api/adresses/{id} - détail d'une adresse
- PUT /api/adresses/{id} - modification d'une adresse
- DELETE /api/adresses/{id} - suppression (après confirmation côté frontend)
- POST /api/adresses/{id}/photo - upload / remplacement de l'image associée
- GET /api/adresses/{id}/export - données formatées pour la génération du PDF (adresse, position, QR code)

Règles métier

- Un utilisateur ne peut posséder plus de 4 adresses : la vérification est effectuée côté serveur avant toute création, indépendamment du frontend.
- Une adresse doit être unique pour un utilisateur donné (mêmes pays/ville/quartier/rue/numéro).
- Un utilisateur ne peut consulter, modifier ou supprimer que ses propres adresses (vérification de propriété systématique).
- Les images uploadées sont validées (type, taille) et stockées via une abstraction de stockage (système de fichiers conteneurisé ou stockage objet compatible S3).

Cas limites à gérer

- Tentative de création d'une 5^e adresse → 409 Conflict avec message explicite.
- Tentative d'accès à l'adresse d'un autre utilisateur → 403 Forbidden.
- Upload d'un fichier non conforme (type, taille) → 400 Bad Request.
- Adresse inexistante → 404 Not Found.

2.3 Microservice Administration & Support (admin-service)

Objectif

Offrir une vision centralisée du système pour les administrateurs et gérer les demandes de support client, en s'appuyant sur des appels REST vers auth-service et address-service.

Fonctionnalités

- GET /api/admin/users - liste paginée des utilisateurs, recherche par email/nom, filtres pays/ville.
- GET /api/admin/adresses - liste paginée des adresses, filtres pays/ville/quartier.
- POST /api/support - envoi d'un message de support (formulaire public : nom, email, message).
- GET /api/admin/support - liste paginée des messages reçus, filtrable par statut.
- PATCH /api/admin/support/{id} - mise à jour du statut (traité / non traité).

Règles métier

- Seuls les rôles ADMIN et SUPPORT_AGENT peuvent accéder aux endpoints /api/admin/**, avec des permissions différenciées (le SUPPORT_AGENT ne gère que le support, pas les utilisateurs).
- Les listes d'utilisateurs et d'adresses affichées par l'admin-service sont obtenues par appel REST synchrone vers auth-service et address-service (pas de duplication de données métier).
- Toute indisponibilité d'un service appelé doit être gérée proprement (timeout, message d'erreur clair) plutôt que de faire planter l'admin-service.

2.4 Sécurité transversale (JWT & RBAC)

La sécurité est un sujet transversal aux trois microservices et à la passerelle. Elle repose sur un token JWT signé, émis par l'auth-service, et validé par chaque service (ou par la Gateway selon le choix d'architecture retenu et justifié).

Matrice de rôles (RBAC) - synthèse attendue

Ressource / Action	USER	SUPPORT_AGENT	ADMIN
Gérer son propre profil et ses propres adresses (max 4)	Oui	Oui (son compte)	Oui (son compte)
Consulter la liste des utilisateurs / adresses (admin)	Non	Non	Oui
Consulter et traiter les messages de support	Non	Oui	Oui
Modifier le rôle ou désactiver un compte	Non	Non	Oui

La matrice complète, incluant chaque endpoint et chaque rôle, doit être détaillée dans le dossier de conception (voir section 4).

2.5 API Gateway & communication inter-services

- Une passerelle unique (API Gateway) expose un point d'entrée unique au frontend, conforme aux URLs déjà utilisées par le Mock Server.
- La Gateway route les requêtes : /api/auth/** et /api/users/** → auth-service ; /api/adresses/** → address-service ; /api/admin/** et /api/support/** → admin-service.
- La communication entre admin-service et les deux autres services se fait en REST synchrone (RestClient, WebClient ou OpenFeign - au choix, à justifier).

- La gestion des erreurs et des délais d'attente (timeout) entre services doit être prévue, sans nécessiter un système de résilience complexe (le circuit breaker est un bonus, non obligatoire).

3. Contraintes Techniques

Stack imposée

- Java 21 (LTS)
- Spring Boot 4 (Spring Web, Spring Data JPA, Spring Security, Spring Validation, Spring Actuator)
- Maven (multi-module ou multi-dépôts, au choix, à justifier dans le dossier de conception)
- PostgreSQL 16 - une base de données dédiée par microservice (database-per-service)
- Flyway - toute évolution de schéma doit passer par une migration versionnée
- Spring Security + JWT pour l'authentification et l'autorisation
- springdoc-openapi pour la documentation interactive (Swagger UI) de chaque service
- Docker & Docker Compose pour l'orchestration complète (3 microservices + Gateway + 3 bases PostgreSQL)
- JUnit 5, Mockito et Testcontainers pour les tests unitaires et d'intégration

Architecture logicielle

Chaque microservice applique une architecture Clean / Hexagonale (ou une architecture mixte pragmatique, à condition qu'elle soit justifiée) avec une séparation stricte des responsabilités :

- domain - entités métier, value objects, ports (interfaces), règles métier pures, indépendantes de Spring.
- application - cas d'usage (use cases) / services applicatifs orchestrant le domaine.
- infrastructure - adaptateurs JPA, clients REST vers les autres services, sécurité, configuration.
- web (interfaces) - contrôleurs REST, DTO, mappers, gestion centralisée des exceptions.

Les principes SOLID doivent être visibles dans les choix de conception (interfaces/ports, inversion de dépendances via l'IoC de Spring, responsabilité unique des classes, etc.).

Bonnes pratiques REST imposées

- DTO systématiques en entrée et en sortie des contrôleurs (aucune entité JPA exposée directement).
- Validation des DTO avec Bean Validation (@NotNull, @Email, @Size, etc.).
- Pagination et tri via Pageable / Page, conformes aux paramètres déjà utilisés par le frontend (page, size, sort).
- Filtrage par paramètres de requête (query params) pour les listes (pays, ville, quartier, statut, etc.).
- Gestion centralisée des erreurs via @RestControllerAdvice et un format d'erreur homogène (de type ProblemDetail / RFC 7807).
- Codes de statut HTTP corrects et cohérents (200, 201, 204, 400, 401, 403, 404, 409, 422, 500).
- Aucune régression de contrat : chaque endpoint doit produire exactement les formats attendus par le frontend du Projet 4.

Transactions et intégrité des données

- Les opérations métier critiques (création de compte, création d'adresse avec vérification du quota) sont encapsulées dans des transactions (@Transactional) au niveau de la couche application.
- Les contraintes d'intégrité (unicité email, unicité adresse) sont également garanties au niveau base de données, pas uniquement applicatif.

Exigences non fonctionnelles

- Chaque microservice est conteneurisé indépendamment et expose un healthcheck via Spring Actuator.
- Aucune donnée sensible en clair : mots de passe hashés, secrets (clé JWT, identifiants base de données) injectés via variables d'environnement.
- Logs structurés et exploitables, sans fuite d'information sensible.
- Le système doit pouvoir démarrer intégralement avec une seule commande (docker compose up).

4. Dossier de Conception Attendu

Avant toute ligne de code métier, un véritable dossier de conception doit être produit et validé. Il constitue un livrable à part entière (voir section 6) et doit inclure au minimum :

Diagrammes

- Diagramme de contexte - vue globale du système : frontend findMe, API Gateway, 3 microservices, bases de données, partenaires B2B.
- Diagramme de composants - par microservice, montrant les couches domain / application / infrastructure / web.
- Diagramme de classes - modèle du domaine de chaque microservice (entités, value objects, relations).
- Diagrammes de séquence - au minimum : inscription, connexion avec émission du JWT, création d'une adresse avec vérification du quota, consultation admin agrégée (appel inter-services).
- Modèle de données (MCD/MPD) - schéma relationnel de chaque base PostgreSQL.

Documents de conception

- Matrice RBAC complète (rôle × ressource × action × endpoint).
- Stratégie de sécurité - cycle de vie du JWT (émission, expiration, refresh token, révocation), stockage côté client, protection des endpoints.
- Stratégie de gestion des transactions - périmètre des @Transactional, gestion des erreurs partielles.
- Organisation des packages - arborescence type détaillée pour chaque microservice.
- Conventions REST retenues - nommage des ressources, verbes HTTP, codes de statut, pagination, versionnement de l'API.
- Stratégie de tests - pyramide de tests visée, périmètre des tests unitaires vs tests d'intégration (Testcontainers), objectif de couverture.

Ce dossier peut être réalisé avec l'outil de votre choix (Miro, Excalidraw, draw.io, PlantUML, Mermaid...) et doit être livré au format PDF ou image, accompagné des sources si possible.

5. Missions par Semaine

♦ Semaine 1 - Conception & Architecture

Objectif : poser une conception solide avant d'écrire la moindre logique métier.

Travail :

- Rédiger le dossier de conception (diagrammes, MCD, matrice RBAC, stratégie de sécurité et de tests).
- Découper le backend en 3 microservices (auth-service, address-service, admin-service) et une API Gateway.
- Modéliser les bases de données PostgreSQL (une par service) et rédiger les premiers scripts Flyway.
- Mettre en place le squelette Maven multi-module et un Docker Compose de base (services vides + PostgreSQL).

Livrables : dossier de conception, diagrammes, arborescence du projet, docker-compose initial.

♦ Semaine 2 - Auth-service & Address-service

Objectif : implémenter les deux microservices cœur de métier avec persistance réelle.

Travail :

- Implémenter auth-service : entités, cas d'usage signup/signin/reset password, hashing BCrypt, persistance JPA, endpoints conformes au contrat du Projet 4.
- Implémenter address-service : CRUD complet des adresses, règle des 4 adresses maximum, unicité, upload d'image, pagination et filtrage.
- Écrire les migrations Flyway correspondantes (tables, contraintes, index).
- Démarrer les tests unitaires des cas d'usage critiques (inscription, création d'adresse).

Bonus : jeu de données de démonstration (seed Flyway) pour faciliter les tests manuels.

♦ Semaine 3 - Sécurité, Intégration & Admin/Support

Objectif : sécuriser l'ensemble du système et le rendre pleinement opérationnel.

Travail :

- Implémenter Spring Security + JWT (génération, validation, refresh) et le RBAC via @PreAuthorize.
- Développer admin-service : vues agrégées utilisateurs/adresses (appels REST vers les autres services) et gestion du support client.
- Mettre en place l'API Gateway (routage vers les 3 microservices, validation centralisée du token).
- Basculer le frontend findMe du Mock Server vers le backend réel et valider l'intégration bout en bout, sans aucune modification du code frontend.

♦ Semaine 4 - Qualité, Tests & Mise en production

Objectif : garantir la fiabilité du système et préparer sa mise en production.

Travail :

- Compléter les tests unitaires et écrire des tests d'intégration avec Testcontainers (endpoints critiques, cas limites).
- Documenter chaque microservice avec OpenAPI/Swagger et activer Spring Actuator (healthchecks).
- Finaliser le Docker Compose complet (3 microservices + Gateway + 3 bases PostgreSQL, démarrage en une commande).
- Rédiger la documentation finale (README, guide d'installation, choix techniques, rapport de sécurité).
- Préparer la démonstration technique en vue de la soutenance.

6. Livrables Attendus

Dans le cadre de ce projet, chaque livrable doit être structuré, exploitable et conforme aux standards professionnels. Les éléments suivants sont attendus :

Code	Livrable	Description détaillée
L1	Code Source	Dépôt(s) Git propre(s) et organisé(s) par microservice, architecture claire (séparation des responsabilités, modularité). Minimum 40 commits significatifs au total, reflétant une progression logique du développement.
L2	Dossier de Conception	Diagrammes (contexte, composants, classes, séquences), modèle de données (MCD/MPD), matrice RBAC, stratégie de sécurité et de tests.
L3	Documentation OpenAPI	Swagger UI exposé par chaque microservice, export des spécifications au format JSON/YAML, exemples de requêtes/réponses.
L4	Scripts Flyway	Migrations versionnées pour chaque base de données, incluant un jeu de données de démonstration (seed) pour la soutenance.
L5	Tests	Tests unitaires (logique métier) et tests d'intégration (Testcontainers) sur les endpoints critiques, avec un rapport de couverture.
L6	Collection Postman	Collection mise à jour prouvant l'équivalence exacte entre le backend réel et les contrats définis dans le Projet 4.
L7	Docker Compose	Orchestration complète du système (3 microservices, Gateway, 3 bases PostgreSQL) démarrable en une seule commande.
L8	Documentation (README)	Présentation du projet, instructions d'installation/configuration/lancement, choix techniques et architecture retenue.
L9	Rapport de Sécurité	Description du modèle RBAC, du cycle de vie du JWT, et des principales vulnérabilités traitées (référentiel OWASP, niveau basique).

7. Compétences Évaluées

Les compétences suivantes seront évaluées tout au long du projet :

Compétence	Description
Spring Core / IoC / DI	Maîtrise du conteneur d'inversion de contrôle et de l'injection de dépendances de Spring.
Spring MVC & API REST	Capacité à concevoir des endpoints REST conformes aux bonnes pratiques et à un contrat existant.
Spring Data JPA & persistance	Modélisation des entités, relations, requêtes, et gestion de la persistance avec PostgreSQL.
Validation & DTO	Séparation entités/DTO et validation rigoureuse des données entrantes.
Gestion des exceptions	Gestion centralisée et cohérente des erreurs applicatives et techniques.
Transactions	Maîtrise du périmètre transactionnel et de la cohérence des données.
Spring Security & JWT	Mise en œuvre d'une authentification robuste basée sur des tokens JWT.
RBAC	Conception et implémentation d'un contrôle d'accès basé sur les rôles.

Compétence	Description
Architecture logicielle (Clean/Hexagonale, SOLID)	Séparation des responsabilités, testabilité et respect des principes SOLID.
Tests unitaires & d'intégration	Fiabilisation du code par une stratégie de tests pertinente (JUnit, Mockito, Testcontainers).
Documentation (OpenAPI, Actuator)	Production d'une documentation API exploitable et supervision applicative.
Communication inter-services & microservices	Conception d'un système distribué simple, cohérent et résilient.

8. Grille d'Évaluation

Critère	Points
Architecture & Clean Code (Clean/Hexagonale, SOLID, packages)	20
Sécurité (JWT, RBAC, hashing, gestion des rôles)	15
Fonctionnalités & conformité aux contrats du Projet 4	20
Persistance & gestion des données (JPA, Flyway, transactions)	15
Tests & qualité (unitaires, intégration, couverture)	15
Documentation & livrables (OpenAPI, README, dossier de conception)	15
Total	100

9. Soutenance

Durée : 25 min

Partie 1 - Démo technique (8 min)

- Démarrage complet du système via docker compose up.
- Démonstration via Swagger UI et/ou la collection Postman.
- Scénario utilisateur complet exécuté depuis le frontend findMe connecté au backend réel.

Partie 2 - Questions techniques (10 min)

- Pourquoi ce découpage en microservices ? Quels compromis avez-vous faits ?
- Comment garantes-vous la conformité stricte aux contrats d'API du Projet 4 ?
- Comment fonctionne votre gestion du JWT et du refresh token ?
- Comment avez-vous conçu votre modèle RBAC ?
- Quelle est votre stratégie de tests et quel est votre niveau de couverture ?

Partie 3 - Feedback (7 min)